

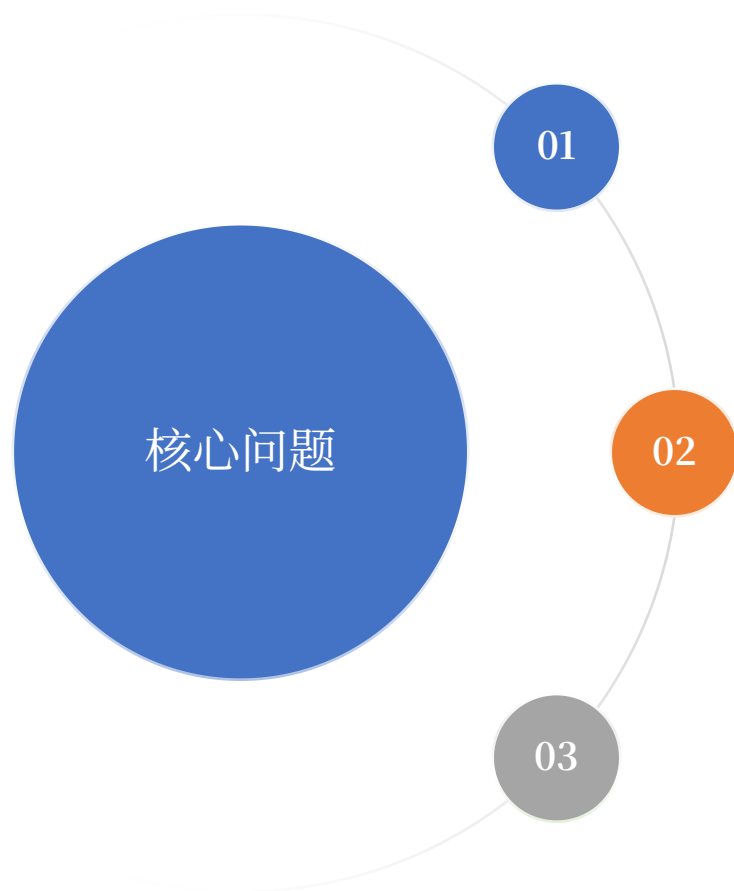
华中科技大学
HUAZHONG UNIVERSITY OF SCIENCE AND TECHNOLOGY



HUST
OPENATOM CLUB

criticality_score: 如何评价最有影响力的开源软件

演讲人：王相智



碎片化与不可持续风险

全球开源项目呈现“小而散”特征，技术路线分散导致生态资源浪费（如物联网领域存在大量重复轮子）

项目生命周期与热度不匹配：84%的代码库存在已知漏洞，部分项目因维护断层快速消亡（如2022年CVE-2023-1017漏洞影响数十亿设备）

安全与合规危机加剧

开源许可证风险显著：2022年1/3代码库存在许可证冲突或未识别问题，合规成本飙升55%

企业对关键依赖项缺乏识别能力，易受供应链攻击（如OpenSSL Heartbleed漏洞暴露核心组件维护脆弱性）

评估体系缺失的决策困境

传统评估依赖主观经验，缺乏量化标准（如企业仅通过“是否知名”选择依赖项）
现有模型（如OpenSSF提出的Criticality Score）侧重单一维度，难以反映项目综合影响力

OpenSSF Criticality Score的短板

数据源单一性

仅依赖GitHub元数据（Star数、Issue响应速度等），无法反映项目在真实技术生态中的渗透度

案例：curl项目在GitHub仅6.8k Star，但被集成到所有主流Linux发行版，传统评分严重低估其影响力

评估维度缺失

忽略语言生态依赖网络（如PyPI/npm包依赖关系）和发行版集成度（如Debian/RedHat默认安装状态）

风险：Log4j因Java生态深度依赖引发全球级漏洞，但OpenSSF评分未预警其关键性

VS

我们Criticality Score的优势

三维动态加权模型

通过Git仓库活性、语言生态渗透度、发行版公共性三大维度动态加权计算，突破单一数据源偏差

生态决策精准化

企业通过语言依赖网络分析，将Python生态中requests（被78%项目依赖）纳入重点监控名单

关键开源软件筛选数据情况

覆盖代码仓库

1000K

统计代码总量

78TB

涉及Linux发行版

11

项目代码行数

17812

涉及软件包生态

11

涉及关基行业

8

目前前100关键开源仓库平台已经上线 GitCode 可信开源频道

可信开源频道是由华中科技大学牵头，联合武汉金银湖实验室、GitCode、中国移动通信有限公司等单位联合研发，旨在通过设计可信开源代码标准体系和评估方法，同时开展开源代码可信检测与修复、开源代码应急响应与影响评估等工作，构建一个安全、可持续且高效的开源生态体系。

全部

Python(可信)

TypeScript(可信)

C++(可信)

JavaScript(可信)

C(可信)

Java(可信)

Go(可信)

PHP(可信)

Ruby(可信)

C#(可信)

综合排序

linux

☆ 22

Linux kernel source tree

C

👤 2

更新于 1 天前

☆ 今日 0

react-native

☆ 22

一个用于构建原生移动应用程序的 JavaScript 库，可以用于构建 iOS 和 Android 应用程序，支持多种原生移动平台，如 iOS, Android, React Native 等。

C++

👤 1

更新于 1 天前

☆ 今日 0

nixpkgs

☆ 4

Nix Packages collection & NixOS

Nix

👤 0

更新于 3 个月前

☆ 今日 0

spring-boot

☆ 12

spring-projects/spring-boot: 是一个用于简化Spring应用开发的框架。适合用于需要快速开发企业级Java应用的项目。特点是可以提供自动配置、独立运行和内置的Tomcat服务器，简化Spring应用的构建和...

Java

👤 5

更新于 2 天前

☆ 今日 0

symfony

☆ 5

symfony/symfony: 是 PHP 的一个开源 Web 框架，提供丰富的组件和工具，可以用于构建大型 Web 应用程序，包括 MVC, ORM, 模板引擎，缓存，安全性等功能。

PHP

👤 1

更新于 2 个月前

☆ 今日 0

eslint

☆ 2

Find and fix problems in your JavaScript code.

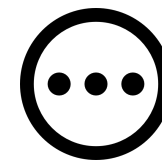
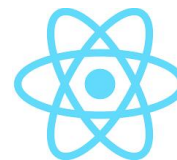
JavaScript

👤 0

更新于 3 个月前

☆ 今日 0

涉及主流开源软件



基于Linux发行版软件包管理器与 deps.dev 计算软件依赖

$$C_{project} = \frac{1}{\sum_i \alpha_i} \sum_i \alpha_i \frac{\log(1 + S_i)}{\log(1 + \max(S_i, T_i))}$$



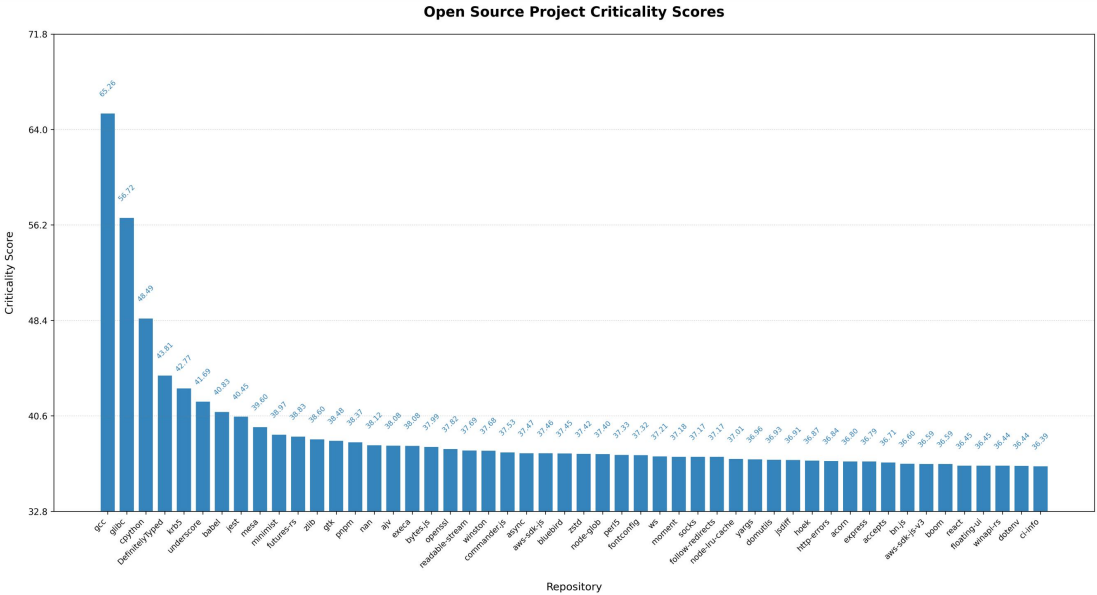
1. Git 元数据:

指标	描述	原因	阈值	权重
created_since	2. 发行版依赖:			
updated_since	指标	描述	原因	
contributor_count	3. 语言生态系统 (npm, pypi):			
org_count	dist_impact	指标	描述	
commit_frequency	pagerank	dist_impact	通过语言生态系统中的依赖比例计算得出，展示其在开发生态系统中的重要性。	
	default_install	pagerank	依赖关系图中每个包的重要性；我们使用 PageRank 算法来计算此指	



模型指标详细展示

评价维度	细分指标
git仓库元数据	创建时间
	更新时间
	贡献者数量
	提交代码的频率
	组织数目
语言生态数据	仓库对应软件在整个语言生态系统中的占比
	仓库对应软件在整个语言生态系统中的page rank算法计算出的比例
发行版生态数据	仓库对应软件在整个发行版生态系统中的占比
	仓库对应软件在整个发行版生态系统中的page rank算法计算出的比例
	发行版软件包下载量
	默认安装的软件



设计理由

追踪开发动态：通过分析提交记录（如创建时间、更新频率），可量化项目的持续维护能力。例如，高频提交可能反映活跃迭代，而长期无更新可能提示项目停滞。

衡量社区参与度：统计贡献者与组织数量，反映项目的开放性和协作规模。如Apache Log4j2漏洞修复中，广泛依赖的组件需社区快速响应，贡献者越多修复效率越高。

01

02

评估方法

归一化处理：对提交频率、贡献者数、Star数等指标设定权重，采用对数归一化或Sigmoid算法消除量纲差异，综合计算项目活跃度得分。

应用场景：识别高价值项目（如长期维护、社区活跃的仓库），辅助企业选型或投资决策。



关键仓库分数分布分析--linux kernel

贡献者

39,318名独立贡献者，反映开源社区活跃度与技术创新驱动力

活跃周期

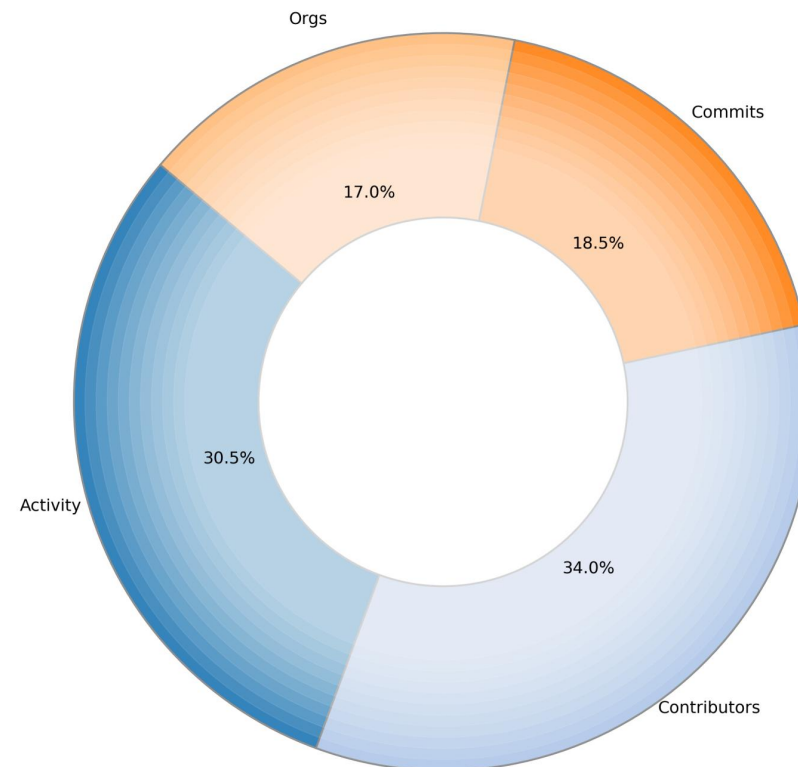
维护跨度：
2005.04-2025.02，
反映项目技术债务管理水平与长期维护能力

提交频率

日均提交量：
1,863次。反映功能迭代速度与缺陷修复响应效率

组织参与

8364个关联技术组织。反映行业生态渗透力与标准化程度





设计理由

评估方法

依赖网络分析：通过获取依赖关系，结合PageRank算法识别核心组件。例如，Java生态中Spring Framework的高依赖占比使其成为关键节点，漏洞影响范围达18%的组件。

生态健康度评估：依赖数量与生态占比（Impact）反映软件的基础性。如Log4j2漏洞影响15%的Java组件，凸显其生态重要性。

风险预警：依赖关系图谱可追踪漏洞传播路径，帮助优先修复关键节点。

数据整合：通过API获取依赖数据，计算Impact值与PageRank得分，1:1加权后归一化处理。

应用场景：识别生态核心组件，指导安全防护与版本升级优先级。



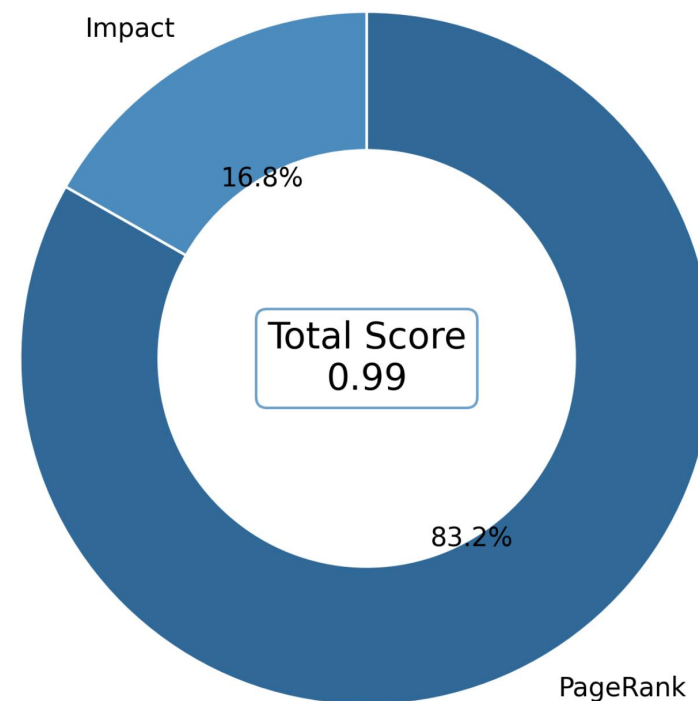
facebook/prop-types 核心指标分析

» 枢纽型组件

- 83.2%的 PageRank占比表明该库处于技术生态关键路径

» 维护必要性

- 高枢纽性带来的安全风险需持续投入维护。



01

设计理由

系统级重要性：Linux发行版（如Debian、Homebrew）的默认包反映基础功能需求。例如，Gentoo的依赖网络可识别构建系统的关键组件。

依赖传播分析：深度优先搜索与PageRank算法量化包在发行版中的影响力。

稳定性保障：发行版通过严格测试确保软件稳定性，映射Gitlink可追溯上游代码质量。

02

评估方法

数据映射：提取发行版元数据，关联Gitlink并聚合依赖数据，通过加权求和实现分数计算。

应用场景：识别核心软件（如默认安装的Nginx、OpenSSL），筛选出对于发行版生态起到关键作用的核心软件。

核心软件下载量：通过对于tunasync中发行版软件包下载量进行统计，分析出核心软件包。

默认安装软件：对于发行版默认安装软件我们给予指定权重，可以凸显出他们在发行版生态中的核心地位



基于GCC编译器项目的依赖关系评估

01

企业级基石：
Debian以37.3%占比主
导企业服务器与云基础设
施

02

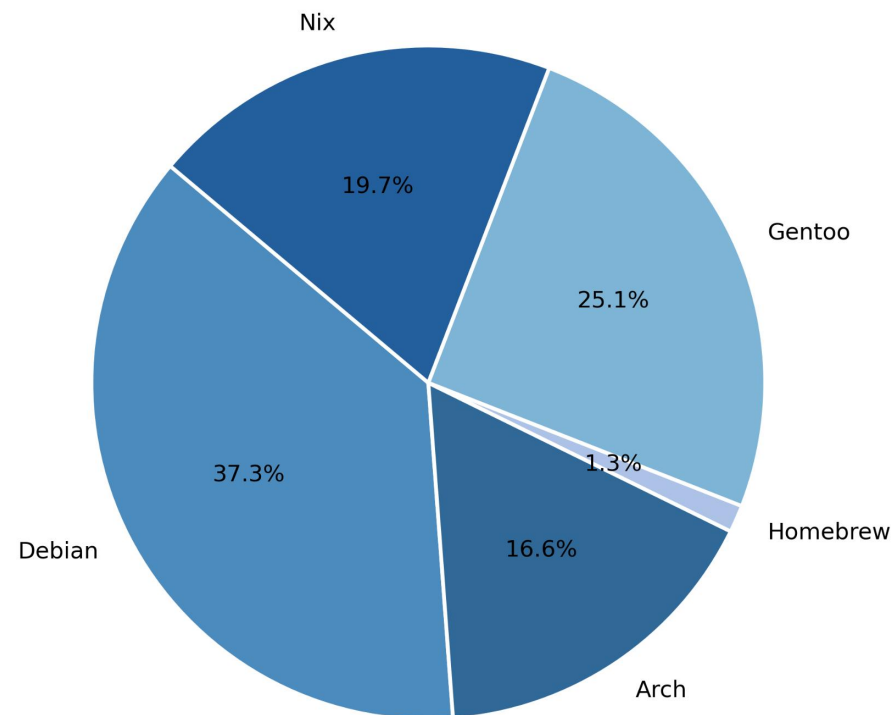
深度定制标杆：
Gentoo 25.1%份额彰
显源码级优化领导力

03

构建革命先锋：Nix
19.7%占比验证声明
式系统创新价值

04

开发者增长极：
Arch+Homebrew双引
擎驱动跨平台工具链演进





多维度综合评价：三大核心指标均以归一化形式参与最终评分，反映项目综合实力。

01



发行版权重凸显：发行版维度得分占比优化，推动成熟项目排名显著提升。

02



社区活跃度验证：Git元数据维度普遍高分，揭示活跃社区对项目可持续性的关键影响。

03



归一化精准识别：将包管理质量量化为语言生态得分的核心权重，消除技术栈差异干扰。

04

Normalized Metrics Trend Analysis of Open Source Projects

